

Chapter 5 Baseline Models

It is important to establish baseline performance on a predictive modeling problem. A baseline provides a point of comparison for the more advanced methods that you evaluate later. In this tutorial, you will discover how to implement baseline machine learning algorithms from scratch in Perl. After completing this tutorial, you will know:

- How to implement the random prediction algorithm.
- How to implement the zero rule prediction algorithm. Let's get started.

5.1 Description

There are many machine learning algorithms to choose from. Hundreds in fact. You must know whether the predictions for a given algorithm are good or not. But how do you know? The answer is to use a baseline prediction algorithm. A baseline prediction algorithm provides a set of predictions that you can evaluate as you would any predictions for your problem, such as classification accuracy or RMSE. The scores from these algorithms provide the required point of comparison when evaluating all other machine learning algorithms on your problem. Once established, you can comment on how much better a given algorithm is as compared to the naive baseline algorithm, providing context on just how good a given method actually is. The two most commonly used baseline algorithms are:

- Random Prediction Algorithm.
- Zero Rule Algorithm. When starting on a new problem that is more difficult than a conventional classification or regression problem, it is a good idea to first devise a random prediction algorithm that is specific to your prediction problem. Later you can improve upon this and devise a zero rule algorithm. Let's implement these algorithms and see how they work.

5.2. Tutorial

This tutorial is divided into 2 parts: baseline performance for your machine learning algorithms.

1. Random Prediction Algorithm.
2. Zero Rule Algorithm. These steps will provide the foundations you need to handle implementing and calculating

5.2.1 Random Prediction Algorithm

The random prediction algorithm predicts a random outcome as observed in the training data. It is perhaps the simplest algorithm to implement. It requires that you store all of the distinct outcome values in the training data, which could be large on regression problems with lots of distinct values. Because random numbers are used to make decisions, it is a good idea to fix the random number seed prior to using the algorithm. This is to ensure that we get the same set of random numbers, and in turn the same decisions each time the algorithm is run. Below is an implementation of the Random Prediction Algorithm in a function named `random_algorithm()`. The function takes both a training dataset that includes output values and a test dataset for which output values must be predicted. The function will work for both classification and regression problems. It assumes that the output value in the training data is the final

column for each row. First, the set of unique output values is collected from the training data. Then a randomly selected output value from the set is selected for each row in the test set.

```
In [1]: use strict;
use warnings;
use Data::Dump qw(dump);
use List::Util qw(zip min max sum uniq);
use sml; # Statistical Machine Learning Library

In [2]: # Example of Making Random Predictions
# Generate random predictions
sub random_algorithm{
    my ($self, $train, $test) = @_;
    my @output_values = map {$_->[-1]} @$train;
    my @unique        = uniq @output_values;
    return [map { $unique[int(rand(@unique))] } @$test]; # predictions
}

sml->add_to_class('random_algorithm', \&{'random_algorithm'});
```

```
Out[2]: *sml::random_algorithm
```

We can test this function with a small dataset that only contains the output column for simplicity. The output values in the training dataset are either 0 or 1, meaning that the set of predictions the algorithm will choose from is (0, 1). The test set also contains a single column, with no data as the predictions are not known.

Running the example calculates random predictions for the test dataset and prints those predictions.

```
In [3]: srand(1);
my $train = [[2,0], [2,1], [2,0], [2,1], [2,0], [2,1], [2,1], [2,1]];
my $test  = [[undef], [undef], [undef], [undef]];
my $predictions = sml->random_algorithm($train, $test);
print dump $predictions;
# Example Output From Making Random Predictions.
# [0, 0, 1, 0]
```

```
[0, 0, 1, 0]
```

```
Out[3]: 1
```

The random prediction algorithm is easy to implement and fast to run, but we could do better as a baseline.

5.2.2 Zero Rule Algorithm

The Zero Rule Algorithm is a better baseline than the random algorithm. It uses more information about a given problem to create one rule in order to make predictions. This rule is different depending on the problem type. Let's start with classification problems, predicting a class label.

Classification

For classification problems, the one rule is to predict the class value that is most common in the training dataset. This means that if a training dataset has 90 instances of class 0 and 10 instances of class 1 that it will predict 0 and achieve a baseline accuracy of 90/100 or 90%. This is much better than the random prediction algorithm that would only achieve 82% accuracy on average. For details on how this is estimate for random search is calculated, see below:

$$= ((0.9 \times 0.9) + (0.1 \times 0.1)) \times 100 = 82\%$$

(5.1)

Below is a function named `zero_rule_algorithm_classification()` that implements this for the classification case.

```
In [4]: # Function To Make Zero Rule Classification Predictions.
# zero rule algorithm for classification
sub zero_rule_algorithm_classification{
  my ($self, $train, $test) = @_;
  my @output_values = map {$_->[-1]} @$train;
  my %counter      = ();
  map {$counter{"$_"}++} @output_values; # Counts
  my $prediction   = $counter{max map {$_} %counter = reverse %counter};
  return [($prediction) x scalar(@$test)]; # predictions
}

sml->add_to_class('zero_rule_algorithm_classification', \&{'zero_rule_algorithm_classi
```

```
Out[4]: *sml::zero_rule_algorithm_classification
```

```
Subroutine sml::zero_rule_algorithm_classification redefined at /usr/local/share/perl5/
5.30/x86_64-linux-thread-multi/sml.pm line 22.
```

The function makes use of the `max()` function on a key attribute, which is obtained from the reversed counter hash. Given a list of class values observed in the training data, the `max()` function takes a set of unique class values and calls the count on the list of class values for each class value in the set. The result is that it returns the class value that has the highest count of observed values in the list of class values observed in the training dataset. If all class values have the same count, then we will choose the first class value observed in the dataset. Once we select a class value, it is used to make a prediction for each row in the test dataset. Below is a worked example with a contrived dataset that contains 4 examples of class 0 and 2 examples of class 1. We would expect the algorithm to choose the class value 0 as the prediction for each row in the test dataset.

Running this example makes the predictions and prints them to screen. As expected, the class value of 0 was chosen and predicted.

```
In [4]: srand(1);
$train = [[2,0], [2,1], [2,0], [2,1], [2,0], [2,1], [2,1] , [2,1]];
$test = [[undef], [undef], [undef], [undef]];
$predictions = sml->zero_rule_algorithm_classification($train, $test);
print dump $predictions;
# Example of Zero Rule Classification Predictions.
# [1, 1, 1, 1]
```

```
[1, 1, 1, 1]
```

```
Out[4]: 1
```

Regression

Regression problems require the prediction of a real value. A good default prediction for real values is to predict the central tendency. This could be the mean or the median. A good default is to use the mean (also called the average) of the output value observed in the training data. This is likely to have a lower error than random prediction which will return any observed output value. Below is a function to do that named `zero rule algorithm regression()`. It works by calculating the mean value for the observed output

values.

$$mean = \sum_{i=1}^n value_i / count(value)$$

(5.2)

Once calculated, the mean is then predicted for each row in the training data.

```
In [5]: # Function To Make Zero Rule Regression Predictions.
# zero rule algorithm for regression
sub zero_rule_algorithm_regression{
  my ($self, $train, $test) = @_;
  my @output_values = map {$_->[-1]} @$train;
  my $prediction     = sprintf '%0.1f', sum(@output_values) / @output_values;
  return [($prediction) x scalar(@$test)]; # predictions
}

sml->add_to_class('zero_rule_algorithm_regression', \&{'zero_rule_algorithm_regression
```

```
Out[5]: *sml::zero_rule_algorithm_regression
```

This function can be tested with a simple example. We can contrive a small dataset where the mean value is known to be 15.

Contrived Regression Dataset And Expected Mean.

10

15

12

15

18

20

mean = (10 + 15 + 12 + 15 + 18 + 20) / 6

mean = 90 / 6

mean = 15

Below is the complete example. We would expect that the mean value of 15 will be predicted for each of the 4 rows in the test dataset.

```
In [6]: srand(1);
$train = [[10], [15], [12], [15], [18], [20]];
$test  = [[undef], [undef], [undef], [undef]];
my $predictions = sml->zero_rule_algorithm_regression($train, $test);
print dump $predictions;
# Example Output From Making Zero Rule Regression Predictions.
# [15.0, 15.0, 15.0, 15.0]
```

```
["15.0", "15.0", "15.0", "15.0"]
```

```
Out[6]: 1
```

5.3 Extensions

Below are a few extensions to the baseline algorithms that you may wish to investigate and implement as an extension to this tutorial.

- Alternate Central Tendency where the median, mode or other central tendency calculations are predicted instead of the mean.
- Moving Average for time series problems where the mean of the last n records is predicted.

5.4 Review

In this tutorial, you discovered the importance of calculating a baseline of performance on your machine learning problem. You now know:

- How to implement a random prediction algorithm for classification and regression problems.
- How to implement a zero rule algorithm for classification and regression problems.